Deep Learning per il Rischio di Credito: stima della PD con PyTorch

Programma commentato — ogni punto, con il suo perché

Stato: versione estesa della sintesi — aggiornata al 4 novembre 2026. **Scopo**: riprendere ogni punto del programma e spiegarlo: che cosa si fa, che cosa significano i termini, e soprattutto perché è stato scelto così. Da usare come documento di lavoro interno o come base discorsiva verso il cliente.

---

1. Il formato, spiegato

4 mattine online, 9:00–13:00, con intervallo di 15 minuti a metà. Sono 240 minuti lordi e 225 netti per mattina, circa 15 ore totali: il perimetro concordato con il cliente. La mattina piena, rispetto alla giornata intera, mantiene alta l'attenzione su contenuti densi e lascia ai partecipanti il pomeriggio operativo.

2 edizioni identiche da 15 analisti quantitativi. In totale 30 persone, ma mai più di 15 per aula: è la soglia oltre la quale un laboratorio guidato online smette di funzionare (il docente non riesce più a seguire chi resta indietro). Le edizioni sono identiche in tutto — materiali, dataset, esercizi — così i due gruppi arrivano allo stesso punto.

Rapporto teoria–pratica ≈ 30–70, senza minutaggi prefissati. Il 70% di pratica è una richiesta del cliente e una scelta didattica: su questi temi si impara scrivendo ed eseguendo codice, non ascoltando. Lo schema ricorrente — apertura con richiamo, tre *pillole* teoriche brevi ciascuna seguita dal proprio laboratorio, sintesi finale — introduce la teoria "appena in tempo": ogni pillola contiene solo ciò che il laboratorio successivo mette subito in pratica. I minutaggi dei singoli blocchi restano liberi per scelta: il rapporto 30–70 si realizza sull'arco dell'intera mattina, e i tempi si adattano alle domande e al ritmo dell'aula.

Esercizi a difficoltà crescente. La platea è omogenea per formazione (tutti STEM, tutti analisti quantitativi) ma eterogenea per livello Python (base, con punte medio-alte). Gli esercizi finali di ogni laboratorio crescono di difficoltà: gli ultimi, opzionali, tengono impegnati i livelli avanzati senza rallentare il gruppo.

Hands-on sui PC dei partecipanti. Ognuno esegue i notebook sulla propria macchina (Visual Studio Code + conda): la competenza che resta dopo il corso è quella esercitata, non quella osservata — e i notebook, interamente commentati in italiano, restano riutilizzabili come punto di partenza per il lavoro reale.

2. L'obiettivo, spiegato

Estendere alle reti neurali la modellazione della PD. La PD (*probability of default*) è la probabilità che una controparte vada in default entro un orizzonte dato; il team la stima già con machine learning classico (XGBoost, Random Forest, alberi decisionali). Il corso non riparte da zero: aggiunge le reti neurali alla cassetta degli attrezzi, concentrandole dove la letteratura e la pratica dicono che aggiungono valore — grandi volumi, variabili categoriche ad alta cardinalità trattate con gli embedding e, soprattutto, le sequenze grezze di transazioni.

I tre requisiti della Vigilanza come filo conduttore. Il cliente è soggetto a controlli attenti e tecnici su tre fronti, che il corso tratta come requisiti di progetto e non come appendici: le **metriche di performance**

(discriminazione, calibrazione, stabilità — mattina 1 e ovunque), l'**esplicabilità** (capire e motivare le scelte del modello — mattina 4, in continuità con lo SHAP già in uso) e il **bias** (la misura quantitativa dell'equità del modello — mattina 4).

XGBoost come champion in ogni laboratorio. *Champion/challenger* è l'approccio in cui il modello in produzione (champion) viene sfidato da un candidato (challenger) sulle stesse metriche e sugli stessi dati. Qui il champion è XGBoost, che il team padroneggia; la rete è il challenger. La scelta ha tre motivi: è la metodologia corretta per adottare un nuovo modello; risponde in anticipo alla domanda tipica della Vigilanza ("perché una rete invece di XGBoost?"); ed è onestà intellettuale, perché la letteratura di benchmark (Grinsztajn et al. 2022; Shwartz-Ziv & Armon 2022; Borisov et al. 2024 — riferimenti in coda) mostra che sul tabellare di dimensioni medie il gradient boosting resta lo stato dell'arte. Il risultato del confronto, qualunque sia, è difendibile: anche quando vince il champion, la tabella documentata è ciò che la banca porta al tavolo del supervisore.

3. Mattina 1 — Oltre l'accuracy, spiegata

Perché si parte dalle metriche. Oggi il team usa solo l'accuracy (la quota di previsioni corrette), che sul default inganna: con un tasso di default del 3%, il modello banale che risponde sempre "nessun default" raggiunge il 97% di accuracy restando inutilizzabile. Tutto il resto del corso valuta i modelli con le metriche introdotte qui, perciò questa mattina è il fondamento.

Le metriche di discriminazione (quanto il modello separa chi andrà in default da chi no): la matrice di confusione (il conteggio di veri/falsi positivi e negativi), la **precision** (quanti, tra i segnalati come rischiosi, lo sono davvero), il **recall** (quanti rischiosi veri vengono trovati), l'**F1** (la loro media armonica); la curva **ROC** con la sua area **AUC** e il **Gini** (= 2·AUC − 1, la forma in cui i validatori riportano la stessa informazione — è la metrica dei report di validazione); la **PR-AUC** (più severa quando le classi sono sbilanciate) e la statistica **KS** (Kolmogorov–Smirnov, la massima distanza tra le distribuzioni dei punteggi delle due classi).

La calibrazione. La PD non è una classe ma una probabilità, e alimenta pricing, accantonamenti e capitale: un modello che discrimina bene ma stima "3%" dove la frequenza osservata è 6% produce prezzi e accantonamenti sbagliati. Strumenti: le curve di calibrazione (*reliability diagram*), il **Brier score** (l'errore quadratico medio sulle probabilità), il **test binomiale** e il **test di Jeffreys** per fascia di rating — quest'ultimo previsto dalle istruzioni BCE per il *validation reporting* dei modelli interni, quindi parte del lessico che il team usa già con il supervisore.

PyTorch essenziale. Tensori, autograd (il calcolo automatico dei gradienti), `nn.Module` e il *training loop* esplicito, presentati in parallelo con il `fit`/`predict` di scikit-learn: per chi viene da lì, la differenza principale è proprio che in PyTorch il ciclo di addestramento si scrive a mano — ed è un vantaggio didattico, perché rende visibile ciò che scikit-learn nasconde.

Il dataset: Taiwan Credit Card Default (UCI). Scelto perché è semplice, pubblico, con uno storico di pagamenti mensili: perfetto per introdurre metriche e primo MLP (*multi-layer perceptron*, il percettrone multistrato — la rete neurale di base) senza complessità di contorno.

4. Mattina 2 — Reti su dati tabellari, spiegata

Perché "fatte bene". Una rete lanciata sul tabellare senza accorgimenti perde contro XGBoost; questa mattina insegna gli accorgimenti, uno alla volta e con esperimenti controllati.

Lo scaling (riportare le variabili su scale comparabili) è necessario per le reti e irrilevante per gli alberi — differenza non ovvia per chi viene da XGBoost: le reti apprendono per gradienti, e scale molto diverse rendono l'ottimizzazione instabile; gli alberi lavorano per soglie, e una soglia non cambia se la variabile è in euro o in migliaia di euro.

Gli embedding sono rappresentazioni numeriche dense *apprese dalla rete* per le variabili categoriche; sostituiscono il one-hot (un vettore con un 1 nella posizione della categoria) quando le modalità sono migliaia — settori merceologici, causali dei movimenti — perché comprimono e fanno emergere somiglianze tra categorie che il one-hot tratta come estranee.

La regolarizzazione combatte l'overfitting (il modello che impara il rumore del campione invece del segnale): il **dropout** spegne a caso una quota di neuroni a ogni passo, la **batch normalization** stabilizza le attivazioni, l'**early stopping** ferma l'addestramento quando la validazione smette di migliorare, il **weight decay** penalizza i pesi grandi.

Lo sbilanciamento (pochi default su molti sani) si gestisce con i **pesi di classe** (gli errori sulla classe rara contano di più) e con la **focal loss**, la variante della cross-entropy che concentra l'apprendimento sugli esempi difficili.

La calibrazione dell'output: le reti tendono a uscire scalibrate, quindi la probabilità va corretta a valle — **Platt scaling** (una regressione logistica sui punteggi), **regressione isotonica** (una correzione monotona non parametrica), **temperature scaling** (un solo parametro che "raffredda" le probabilità).

Il cenno a FT-Transformer e TabNet (architetture nate apposta per il tabellare) dà i riferimenti per approfondire, senza farne il centro del corso. La **discussione sui portafogli low-default** (pochissimi default osservati: il caso Large Corporate) avverte che lì le stime del Gini sono instabili e i test hanno bassa potenza — è il motivo per cui "quando il deep learning ha senso" è un tema esplicito del corso, utile a tutte e tre le strutture in aula.

Il dataset: un campione di Home Credit Default Risk, scelto perché ricco di variabili comportamentali e categoriche ad alta cardinalità — il terreno giusto per gli embedding. Fornito come file pronto: niente registrazioni Kaggle in aula.

5. Mattina 3 — Sequenze di transazioni, spiegata

Perché è il modulo distintivo. Le transazioni sono i predittori comportamentali per eccellenza: vedono il deterioramento *prima* che arrivi sulle rate (stipendio irregolare, saldo in calo, fido saturo — l'evidenza classica è Norden & Weber 2010). Oggi il team le usa già, ma **aggregate** (medie, conteggi, utilizzo su finestre): i livelli entrano nel modello, la *dinamica* no, perché l'aggregazione distrugge l'ordine degli eventi — il pattern "stipendio, poi prelievo massiccio il giorno dopo, ogni mese" sparisce in una media. Dare alla rete la **sequenza grezza** conserva anche la dinamica: è ciò che XGBoost, per costruzione, non può ricevere in input, e quindi il territorio dove il vantaggio delle reti è strutturale.

La rappresentazione. Una sequenza di movimenti diventa un tensore; il **padding** allunga le sequenze corte con posizioni fittizie perché abbiano tutte la stessa lunghezza, e il **masking** dice alla rete di ignorarle.

Le architetture. Le **RNN** (reti ricorrenti) leggono la sequenza un passo alla volta mantenendo uno stato; **LSTM** e **GRU** sono le varianti che gestiscono le dipendenze lunghe senza "dimenticare"; le **reti convoluzionali 1D** cercano motivi locali lungo la sequenza; il cenno ad **attention e Transformer** indica dove va la ricerca.

Il confronto misurato. A parità di informazione, tre modelli: LSTM sulle sequenze contro MLP e XGBoost sugli stessi dati aggregati — il valore aggiunto della dinamica si legge in punti di Gini, non in dichiarazioni.

Il dataset: transazioni sintetiche costruite ad hoc. Scelta obbligata e vantaggiosa insieme: i dati reali della banca sono esclusi dall'aula (riservatezza, GDPR, iter di autorizzazione; anche anonimizzati, i pattern di spesa restano re-identificabili) e dataset pubblici di transazioni bancarie realistiche non esistono, per gli stessi vincoli. Il sintetico offre zero rischio privacy, struttura che replica conti e carte della banca, segnale comportamentale iniettato ad arte (difficoltà didattica calibrata) e **verità nota**: si può verificare se il modello — e le spiegazioni della mattina 4 — ritrovano il pattern vero. Limite dichiarato: il sintetico illustra il metodo, non dimostra le performance sul portafoglio reale; la validazione sui dati della banca avviene dopo, internamente.

L'appendice GPU. Tutti i laboratori girano su CPU in pochi minuti (così nessuno resta indietro per motivi hardware); la GPU è un capitolo a parte perché qui, con l'LSTM, i tempi iniziano davvero a pesare. Il notebook autonomo verifica **CUDA** (la piattaforma NVIDIA che permette a PyTorch di usare la scheda grafica), riesegue il training misurando lo **speedup** (il rapporto tra tempo su CPU e su GPU) ed esiste in versione **Google Colab** (GPU T4 gratuita) per chi non ha una scheda locale. Demo dal vivo del docente in chiusura.

6. Mattina 4 — Esplicabilità, bias e governance, spiegata

Da SHAP a Captum. Il team usa già SHAP e feature importance sul champion: **Captum**, la libreria di interpretabilità dell'ecosistema PyTorch, è il ponte naturale. **DeepSHAP** e **GradientSHAP** sono le varianti dei valori di Shapley adattate alle reti; **Integrated Gradients** attribuisce l'importanza integrando i gradienti lungo il percorso da un input di riferimento all'input reale. Il laboratorio confronta le attribuzioni di Captum sul MLP con quelle di SHAP su XGBoost *sulle stesse osservazioni* — così il team vede continuità, non un mondo nuovo — e mostra sulle sequenze dell'LSTM quali movimenti pesano sulla PD (ed essendo il segnale sintetico noto, si verifica che le spiegazioni puntino al pattern vero). Si tratta anche la **stabilità delle spiegazioni** (tra campioni e nel tempo): una spiegazione che cambia a ogni estrazione non convince nessun validatore.

I vincoli di monotonicità impongono relazioni di buon senso economico — ad esempio: all'aumentare dell'utilizzo del fido, la PD non può diminuire — e sono molto apprezzati dalla Vigilanza perché rendono il modello prevedibile nelle direzioni che contano.

La fairness quantitativa misura il bias con metriche: la **demographic parity** chiede tassi di esito simili tra gruppi, l'**equalized odds** chiede tassi di errore simili. Con **Fairlearn** si fa un audit su un attributo protetto sintetico, si applica una mitigazione e se ne misura il costo in performance: la fairness è un trade-off da governare, non uno slogan.

La governance. Il **PSI** (*Population Stability Index*) misura quanto la popolazione corrente si è spostata da quella di sviluppo (soglie di prassi: 0,10 e 0,25) ed è lo strumento del monitoraggio. La distinzione **ricalibrazione** (si correggono le probabilità, il modello resta) / **retraining** (si riaddestra) / **challenger** (si sostituisce) è la traduzione operativa del "fine-tuning" che la Vigilanza a volte richiede: sapere quale dei tre serve, e documentarlo, è metà della risposta. Cenno alla PD *lifetime* richiesta da IFRS 9 per i crediti in stage 2, come estensione oltre l'orizzonte a 12 mesi.

La scheda di validazione. L'esercizio di sintesi finale compila un template di scheda di validazione del modello con le metriche, le spiegazioni e i test prodotti nelle quattro mattine: è il documento-tipo del dialogo con la Vigilanza, e il template resta alla banca, riusabile.

7. Tecnologie e materiali, spiegati

- **PyTorch (build CPU)**: il framework di deep learning del corso — MLP, embedding, LSTM. La build CPU garantisce che tutto giri su qualunque postazione.
- **scikit-learn**: preprocessing, baseline (regressione logistica), metriche, calibrazione.
- **XGBoost**: il champion di ogni confronto.
- **SHAP / Captum / Fairlearn**: spiegazioni del champion, spiegazioni delle reti, fairness.
- `environment.yml`: la "ricetta" dell'ambiente conda, con **versioni fissate** — tutti i partecipanti delle due edizioni lavorano nello stesso ambiente, azzerando il "da me funziona".
- **Notebook 0 di setup, in apertura della prima mattina**: crea l'ambiente, verifica gli import, lancia un mini-training di prova (~30 secondi). Si esegue insieme in aula perché una verifica pre-corso non è praticabile; va riservato il tempo necessario.
- **Dataset come file pronti (CSV/Parquet)**: scaricare Home Credit da Kaggle richiederebbe account e autenticazione API — una fonte classica di perdite di tempo in aula.
- **Soluzioni degli esercizi a fine corso**: consegnarle prima svuoterebbe gli esercizi a difficoltà crescente della loro funzione.

8. Le ipotesi di lavoro, spiegate

Su soglie, definizione di default e orizzonte il cliente non ha dato indicazioni: si adotta la **prassi europea corrente**, dichiarandola in aula come tale ("prassi, non norma"), con la **funzione di validazione interna** chiamata a confermarla prima dell'avvio — la sostituzione è un intervento minimo sui materiali.

- **Gini a semaforo** (sotto 40% debole, 40–60% accettabile, sopra 60% buono): lo schema diffuso nella pratica di validazione, con l'avvertenza low-default.
- **PSI 0,10 / 0,25**: le soglie classiche di stabilità/deriva.
- **Test binomiale e di Jeffreys per fascia di rating**: il secondo previsto dalle istruzioni BCE per il *validation reporting*; Hosmer–Lemeshow come alternativa storica.
- **Default: art. 178 CRR** (*Capital Requirements Regulation*): 90 giorni di arretrato rilevante (*past due*) oppure improbabile adempimento (*unlikely to pay*), con le soglie di rilevanza EBA (100 € retail, 500 € non retail, 1% relativo).
- **Orizzonte: PD a 12 mesi**, lo standard dei modelli **IRB** (*Internal Ratings-Based*, i modelli interni autorizzati per il capitale); la PD *lifetime* di IFRS 9 è citata come estensione.
- I dataset sintetici sono **etichettati secondo queste definizioni**, così il vocabolario d'aula coincide con quello usato con la Vigilanza.

Restano aperti: la conferma delle ipotesi da parte della validazione interna, la verifica che le postazioni possano creare ambienti conda da internet (proxy e permessi aziendali), il calendario delle due edizioni, il tempo da riservare al Notebook 0 nella prima mattina.

Riferimenti bibliografici

- Grinsztajn L., Oyallon E., Varoquaux G. (2022), "*Why do tree-based models still outperform deep learning on typical tabular data?*", Advances in Neural Information Processing Systems 35 (NeurIPS 2022), Datasets and Benchmarks Track — arXiv:2207.08815.
- Shwartz-Ziv R., Armon A. (2022), "*Tabular data: Deep learning is not all you need*", Information Fusion, 81, 84–90 — arXiv:2106.03253.

- Borisov V., Leemann T., Seßler K., Haug J., Pawelczyk M., Kasneci G. (2024), "*Deep Neural Networks and Tabular Data: A Survey*", IEEE Transactions on Neural Networks and Learning Systems, 35(6), 7499–7519 — DOI 10.1109/TNNLS.2022.3229161.
- Norden L., Weber M. (2010), "*Credit Line Usage, Checking Account Activity, and Default Risk of Bank Borrowers*", The Review of Financial Studies, 23(10), 3665–3699.